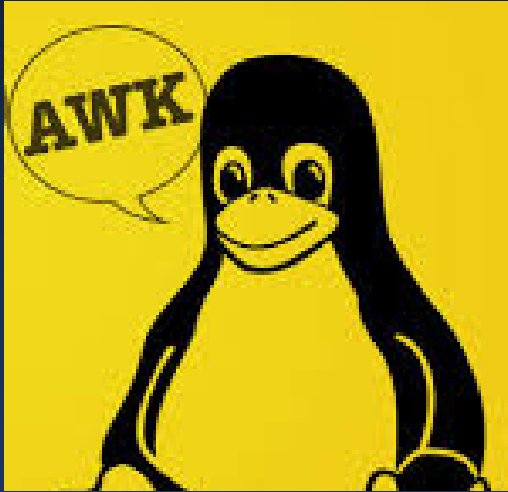




Higher Diploma in Computer Science

Computer Systems & Networks





```
$ awk 'pattern { action }'
```

built-in
variables

Built in Variables

- ARGC Retrieves the number of passed parameters.
- ARGV Retrieves the command line parameters.
- ENVIRON Array of the shell environment variables and corresponding values.
- FILENAME The file name that is processed by awk.
- IGNORECASE To ignore the character case.

Built in Variables

- Use of NR (display all fields for all records *incl* line number)

\$ awk '{print NR,\$0}' employee.txt

- Use of NF (Display emp Name field and also the last field)

\$ awk '{print \$2,\$NF}' employee.txt

\$ awk 'NR==3, NR==6 {print NR,\$0}' employee.txt

What does this last command do?

Be careful with Syntax

\$ awk '\$1 ~ /^[CF]/ {print \$0}' employee.txt

awk '\$1 ~ /^[C,F]/ {print \$0}' employee.txt matches any string starting with C, F, or a comma.

- For each line, checks if the **first field** (\$1) starts with C or F.

If yes, prints the **entire line** (\$0).

\$awk '\$1 ~ /^[CF]/ {print NR, \$0}' employee.txt

#without the comma awk will just concatenate the line number and the line without spacing

- For each line where the first field (\$1) starts with C or F, it prints:

The line number (NR)

The full line (\$0)

Built in Variables

Field separators

- The whitespace character (space or tab) is default
- You can specify it using **FS=“*other_delimiter*”, recall:**

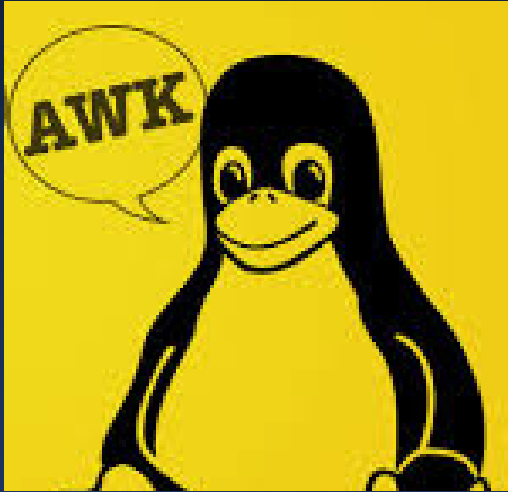
```
$awk 'BEGIN {  
    FS = ","  
    OFS = "-"  
}  
{  
    print $2  
}' employee.txt
```

#control output formatting too

User defined Variables

```
compsys@compsys-virtualbox:~$ awk 'BEGIN { test="Here we are now"  
> print test  
> }'  
Here we are now  
compsys@compsys-virtualbox:~$ █
```

- similar to variables in bash scripting, no need to define



```
$ awk 'pattern { action }'
```

redirection

awk redirection using pipes

- **awk** can receive input and send output to other commands via the pipe.

```
compsys@compsys-virtualbox:~$ TEST="Wow it's mid September"
compsys@compsys-virtualbox:~$ OUTPUT=`echo $TEST | awk '{print $4}'`
compsys@compsys-virtualbox:~$ echo $OUTPUT
September
compsys@compsys-virtualbox:~$ OUTPUT=`echo $TEST | awk '{print $NF}'`
compsys@compsys-virtualbox:~$ echo $OUTPUT
September
```

\$(NF) is the last field

Try out: **\$ awk '/[Mm]anager/ { print \$2, \$NF }' employee.txt**

awk redirection using > and >>

As previously:

- Using > will redirect output to a new file
 - Remember it will overwrite if the file exists
- Using >> will append output to a new file/the end of an existing file



BE CAREFUL OF YOUR CHOICE

ps aux

- monitor processes running on your Linux system

```
$ ps aux
```

- It can be large to redirect to a file

```
$ ps aux > ActiveProcesses.txt
```

- View the **header** of the **ps aux** output plus one output line to see what this command will show about active processes:

```
$ cat ActiveProcesses.txt | head -n 2
```

ps aux

- You can filter which records to execute actions on through the **awk** powerful pattern matching feature.

```
$ ps aux > ActiveProcesses.txt
```

```
$ cat Acti* | head -n 2          #look at ps aux output
```

```
$ awk '/firefox/ { print $1, $3 }' ActiveProcesses.txt
```

Amend this to check the time the processes was started

grep and *pipe* to return a process using *ps aux*

EXERCISE: using **ps** and **grep**, return details for a *firefox* process

```
compsys@compsys-virtualbox:~$ ps aux | grep 'firefox'
compsys      1876  17.0   2.8 343056 57376 ?        D   12:56   0:01 /snap/firefox/1810/usr/lib/firefox/firefox
compsys      2019  14.5   2.0 339248 41048 ?        D   12:56   0:00 /snap/firefox/1810/usr/lib/firefox/firefox
compsys      2021   0.0   0.0   636     4 pts/0    R+   12:56   0:00 grep --color=auto firefox
```

PROBLEM:

O/p lists processes but also lists the **grep** command itself

\$grep -v grep *# exclude grep from the output*

```
~$ ps aux | grep 'firefox' | grep -v grep
```

EXERCISE: List the timestamp of all processes that are *firefox* related:

```
~$ ps aux | grep 'firefox' | grep -v grep | awk '{print $9}'
```

Find specifics within the */etc/passwd* file

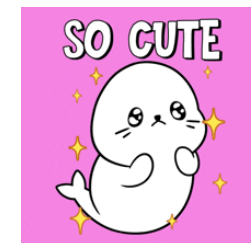
You want to print the username and the user's shell details *only* for the first seven lines of the system **/etc/passwd** file:

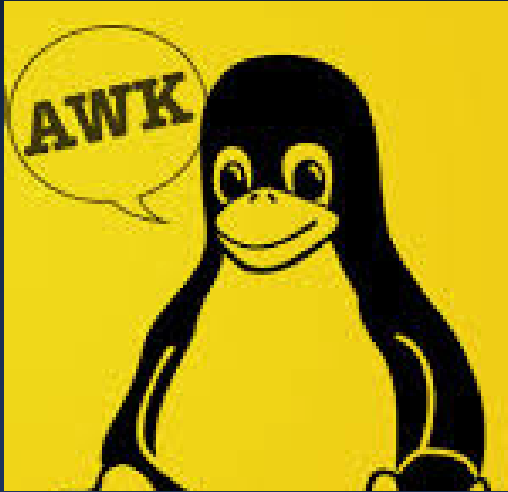
```
awk -F: '{print $1, $7}' /etc/passwd | head -7
```

/etc/passwd is colon separated

Output could be formatted as a table, recall **printf**:

```
awk -F: '{printf "%-20s %-30s\n", $1, $7}' /etc/passwd | head -7
```





```
$ awk 'pattern { action }'
```

Further commands

Using other commands: *grep* and *cut*

EXERCISE: you wanted to return the line number of a matching pattern *userinput* from *employee.txt*:

FIRST: THINK LOGICALLY

- **grep** to pattern match and then
- **cut** to return the desired section (line no. only)

```
$grep -in "${userinput}" employee.txt | awk '{print $2}'
```

```
$grep -in "${userinput}" employee.txt | cut -f1 -d":"
```



EXERCISE: Count the number of lines matching

Variables Syntax:

```
num_matches=`grep -c Caroline employee.txt`  
echo "Caroline appears in $num_matches lines."
```

Backticks assign the grep output to the variable num_matches

Or, more modern syntax (recommended for readability and nesting):

```
num_matches=$(grep -c Caroline employee.txt)  
echo "Caroline appears in $num_matches lines."
```

EXERCISE: Count the number of lines NOT matching

- reverse grep i.e. the following Counts the number of lines NOT matching

num_matches=\$(grep -v Caroline employee.txt)

PROBLEM???

comma-separated values (.csv) files

```
compsys@compsys-virtualbox:~$ cat employee.csv
100,frank walsh,manager,sales,€5000
200,caroline cahill,developer,technology,€5500
```

- Print the first column of this csv file

```
compsys@compsys-virtualbox:~$ awk '{print $1}' employee.csv
ID,name,title,dept,wages
100,frank
200,caroline
```

- Amend to print our names:

```
compsys@compsys-virtualbox:~$ awk -F"," '{print $2}' employee.csv
name
frank walsh
caroline cahill
```

Built-in variable OFS.

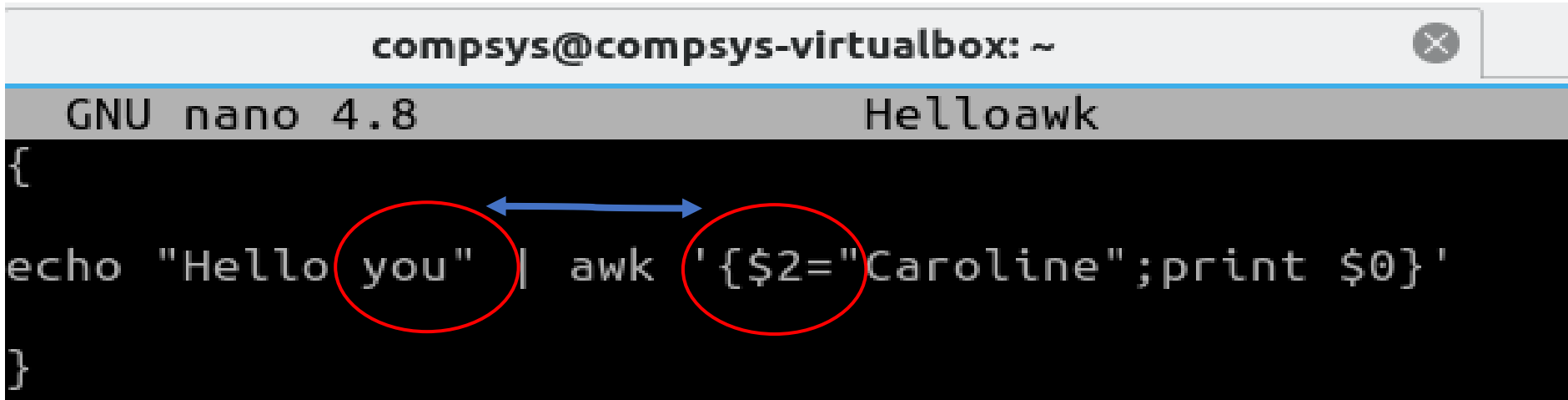
- OFS (Output field separator) is used to separate the output fields.

```
compsys@compsys-virtualbox:~$ sed 's/ /,/g' employee* > emp.csv
compsys@compsys-virtualbox:~$ cat emp.c*
#!/bin/bash
#,This,is,a,file,that,demonstrates,the,employees,details,

300,frank,manager,sales,€5000
400,caroline,developer,technology,€5500
100,mary,sysadmin,technology,€7000
200,rosanne,manager,marketing,€9500
500,eamonn,dba,technology,46000
600,colm,sysadmin,technology,€4500
There,are,no,$,currency,only,€
compsys@compsys-virtualbox:~$ awk -F"," 'NR!=1{print $2, $4}' OFS=":" emp.csv
```

Multiple Commands

- separate with a semicolon



A terminal window titled 'compsys@compsys-virtualbox: ~' with a close button. The window shows the GNU nano 4.8 editor with a file named 'Helloawk'. The code inside the editor is:

```
{  
echo "Hello you" | awk '{$2="Caroline";print $0}'  
}
```

 The words 'you' and 'Caroline' are circled in red. A blue double-headed arrow points from the circle around 'you' to the circle around 'Caroline', indicating the replacement of the second field.

Hello Caroline

awk Control Statements

- '&&' means 'logical AND'
- '||' means 'logical OR',
- '!' means 'logical NOT'

Special Symbols

if (condition) statement [else statement]

while (condition) statement

do statement while (condition)

for (expr1; expr2; expr3) statement

exit [expression]

for (var in array) statement

break

continue

IF statement

```
# This re-displays the data entered so the user can confirm.
echo "----- Entered Data -----"
echo $name "; " $address "; " $mobileNumber "; " $emailAddress ";"
echo -n "Are you sure you want to add this contact (y/n) :"
read answer
if [[ "$answer" == "y" || "$answer" == "Y" ]]
then
    echo $name";"$address";"$mobileNumber";"$emailAddress";" "added successfully"
    echo $name";"$address";"$mobileNumber";"$emailAddress";" >> contactdetails.txt
else
    echo "Contact has not been added"
fi
```

String functions

awk 'BEGIN { x = "caroline"; print toupper(x) }'

```
compsys@compsys-virtualbox:~$ awk 'BEGIN{x = "caroline"; print toupper(x) }'  
CAROLINE  
compsys@compsys-virtualbox:~$ awk 'BEGIN{  
> x="Caroline";  
> print toupper(x)  
> }'  
CAROLINE
```

**tolower()
toupper()**

length() in-built function

If no argument is supplied, **length()** returns the length of **\$0**

- Use arithmetic comparisons when suitable:

```
awk -F, 'length($1)==5 && (length($2)==10 || length($2)==15)  
&& length($3)==30'
```

\$man awk

Functions

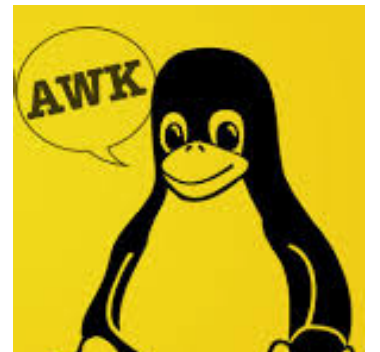
```
compsys@compsys-virtualbox: ~  
GNU nano 4.8 passwordFn.sh  
#!/bin/bash  
#Have a look and see how this uses a function  
#you've created to return each users home path  
#and included \n to move to new line  
awk '  
function myfunc()  
{  
printf "User %s has home path at %s\n", $1,$6  
}  
BEGIN{FS=":"}  
{  
myfunc()  
}' /etc/passwd
```

awk Summary

awk is going to break each line into “fields” based on whitespace and ignores leading or trailing whitespace.

Also, **awk** is automatically going to set a bunch of useful variables for you:

- **NF** – how many fields in the current line
- **NR** – what the current line number is
- **\$1, \$2, \$3 .. \$9 ..** –value of each field on the current line
- **\$0** – the content of the current line



00: Module introduction



Linux · Computer Networks

01: Intro and Command Line...



Scripting · Permissions ·
Navigation

02: File and Text Manipulation



Redirection · grep ·
Positional Param · Exit
Codes

03: Loops, Menus, sed

MAIN - MENU

1. User Management
2. Service Management
3. Process Management
4. Backup

Enter your choice [1-4]

case · select · loops · sed